

The normal distribution

In this lab, you'll investigate the probability distribution that is most central to statistics: the normal distribution. If you are confident that your data are nearly normal, that opens the door to many powerful statistical methods. Here we'll use the graphical tools of R to assess the normality of our data and also learn how to generate random numbers from a normal distribution.

Getting Started

Load packages

In this lab, we will explore and visualize the data using the **tidyverse** suite of packages as well as the **openintro** package.

Let's load the packages.

```
library(tidyverse)
library(openintro)
```

The data

This week you'll be working with fast food data. This data set contains data on 515 menu items from some of the most popular fast food restaurants worldwide. Let's take a quick peek at the first few rows of the data.

Either you can use `glimpse` like before, or `head` to do this.

```
library(tidyverse)
library(openintro)
data("fastfood", package='openintro')
head(fastfood)
```

```
## # A tibble: 6 x 17
##   restaurant item      calories cal_fat total_fat sat_fat trans_fat cholesterol
##   <chr>      <chr>      <dbl>  <dbl>    <dbl>  <dbl>    <dbl>      <dbl>
## 1 Mcdonalds  Artisan G~    380    60      7      2      0         95
## 2 Mcdonalds  Single Ba~    840   410     45     17     1.5       130
## 3 Mcdonalds  Double Ba~   1130   600     67     27      3       220
## 4 Mcdonalds  Grilled B~    750   280     31     10     0.5       155
## 5 Mcdonalds  Crispy Ba~    920   410     45     12     0.5       120
## 6 Mcdonalds  Big Mac      540   250     28     10      1        80
## # i 9 more variables: sodium <dbl>, total_carb <dbl>, fiber <dbl>, sugar <dbl>,
## #   protein <dbl>, vit_a <dbl>, vit_c <dbl>, calcium <dbl>, salad <chr>
```

You'll see that for every observation there are 17 measurements, many of which are nutritional facts.

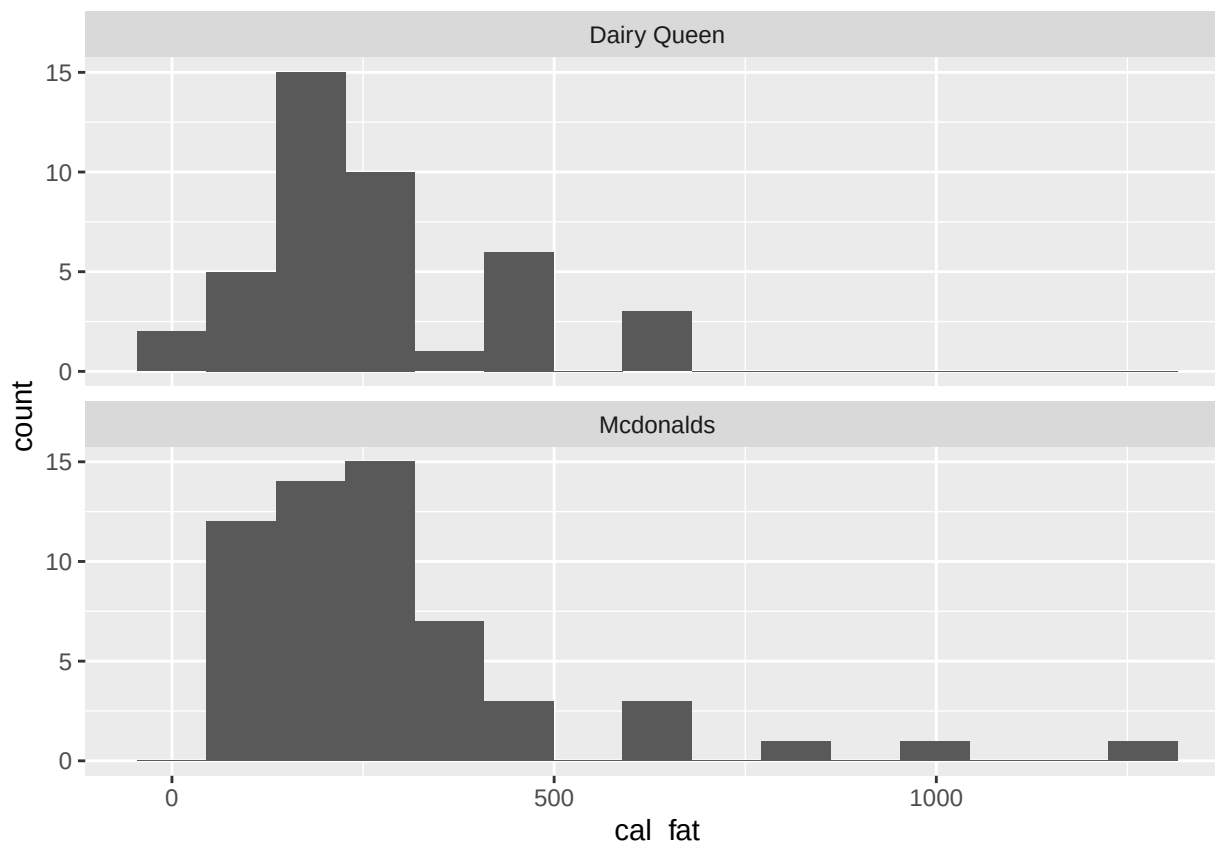
You'll be focusing on just three columns to get started: restaurant, calories, calories from fat.

Let's first focus on just products from McDonalds and Dairy Queen.

```
mcdonalds <- fastfood %>%
  filter(restaurant == "Mcdonalds")
dairy_queen <- fastfood %>%
  filter(restaurant == "Dairy Queen")
```

1. Make a plot (or plots) to visualize the distributions of the amount of calories from fat of the options from these two restaurants. How do their centers, shapes, and spreads compare?

```
ggplot(bind_rows(mcdonalds, dairy_queen), aes(x = cal_fat)) +
  geom_histogram(bins = 15) +
  facet_wrap(~ restaurant, ncol = 1)
```



```
bind_rows(mcdonalds, dairy_queen) %>%
  group_by(restaurant) %>%
  summarise(mean = mean(cal_fat), median = median(cal_fat),
            sd = sd(cal_fat), IQR = IQR(cal_fat),
            min = min(cal_fat), max = max(cal_fat))
```

```
## # A tibble: 2 x 7
##   restaurant mean median    sd  IQR  min  max
##   <chr>      <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 Dairy Queen 260.   220  156.  150    0   670
## 2 McDonalds  286.   240  221.  160   50  1270
```

Both distributions are unimodal and right skewed. Their centers are close, but the mean and median values for McDonald's are slightly larger. The mean is 286, and the median is 240 calories of fat for McDonald's, whereas the mean and median for Dairy Queen are 261 and 220, respectively. McDonald's has a longer tail, with a tail of its histogram going all the way to 1,270 calories of fat, whereas Dairy Queen's tail ends at around 670 calories.

The normal distribution

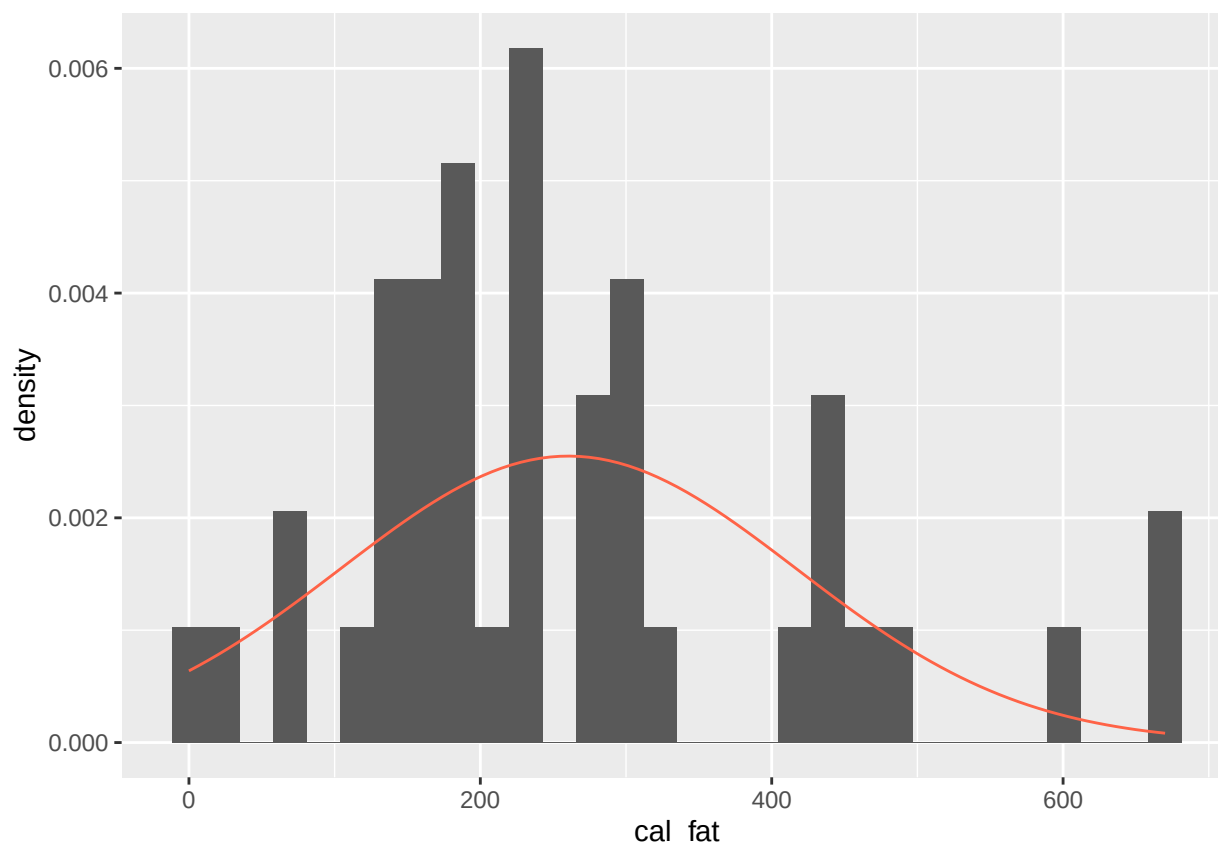
In your description of the distributions, did you use words like *bell-shaped* or *normal*? It's tempting to say so when faced with a unimodal symmetric distribution.

To see how accurate that description is, you can plot a normal distribution curve on top of a histogram to see how closely the data follow a normal distribution. This normal curve should have the same mean and standard deviation as the data. You'll be focusing on calories from fat from Dairy Queen products, so let's store them as a separate object and then calculate some statistics that will be referenced later.

```
dqmean <- mean(dairy_queen$cal_fat)
dqsd   <- sd(dairy_queen$cal_fat)
```

Next, you make a density histogram to use as the backdrop and use the `lines` function to overlay a normal probability curve. The difference between a frequency histogram and a density histogram is that while in a frequency histogram the *heights* of the bars add up to the total number of observations, in a density histogram the *areas* of the bars add up to 1. The area of each bar can be calculated as simply the height *times* the width of the bar. Using a density histogram allows us to properly overlay a normal distribution curve over the histogram since the curve is a normal probability density function that also has area under the curve of 1. Frequency and density histograms both display the same exact shape; they only differ in their y-axis. You can verify this by comparing the frequency histogram you constructed earlier and the density histogram created by the commands below.

```
ggplot(data = dairy_queen, aes(x = cal_fat)) +
  geom_blank() +
  geom_histogram(aes(y = ..density..)) +
  stat_function(fun = dnorm, args = c(mean = dqmean, sd = dqsd), col = "tomato")
```



After initializing a blank plot with `geom_blank()`, the `ggplot2` package (within the `tidyverse`) allows us to

add additional layers. The first layer is a density histogram. The second layer is a statistical function – the density of the normal curve, `dnorm`. We specify that we want the curve to have the same mean and standard deviation as the column of fat calories. The argument `col` simply sets the color for the line to be drawn. If we left it out, the line would be drawn in black.

2. Based on the this plot, does it appear that the data follow a nearly normal distribution?

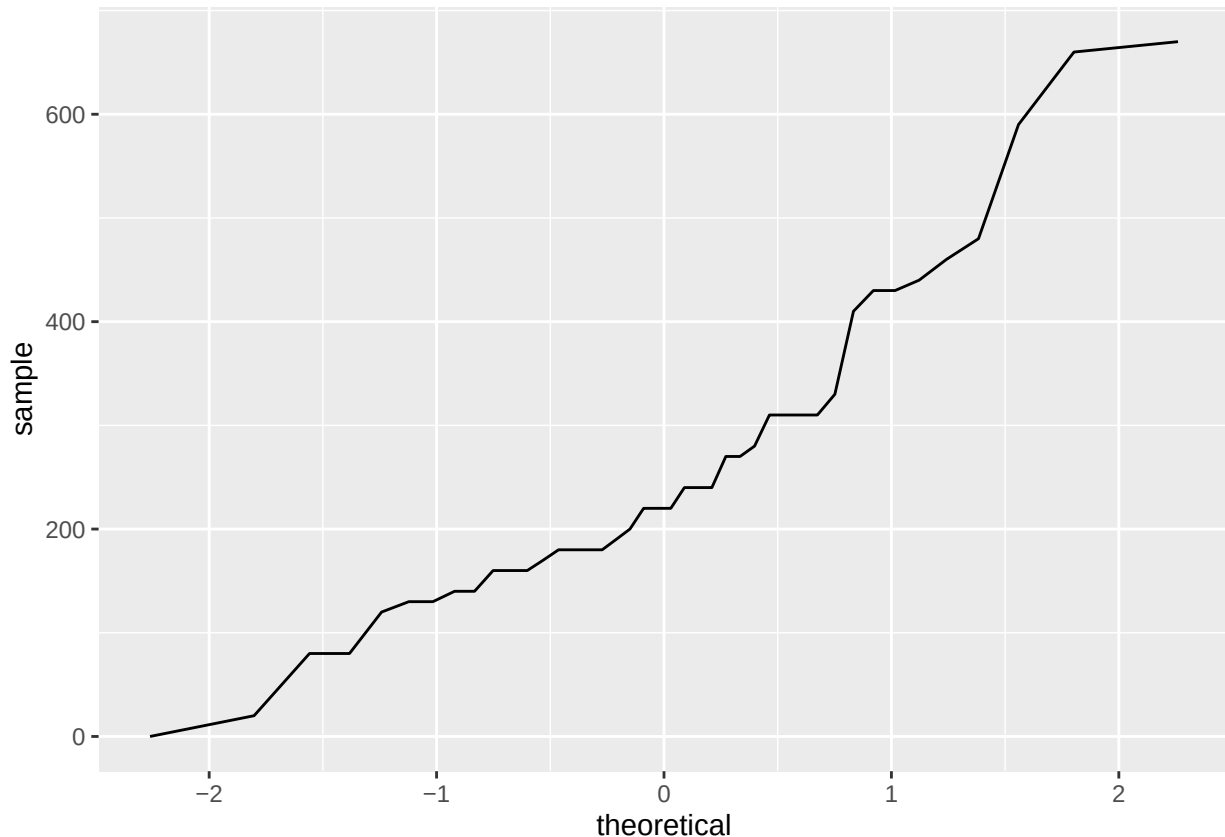
Insert your answer here

No. Although there is one peak in the histogram, the shape is right-skewed, and the bars do not fit a normal distribution curve, since the values in the histogram are clustered on the left and spread out in the long tail to the right, while the probabilities in the curve are below zero.

Evaluating the normal distribution

Eyeballing the shape of the histogram is one way to determine if the data appear to be nearly normally distributed, but it can be frustrating to decide just how close the histogram is to the curve. An alternative approach involves constructing a normal probability plot, also called a normal Q-Q plot for “quantile-quantile”.

```
ggplot(data = dairy_queen, aes(sample = cal_fat)) +  
  geom_line(stat = "qq")
```



This time, you can use the `geom_line()` layer, while specifying that you will be creating a Q-Q plot with the `stat` argument. It's important to note that here, instead of using `x` instead `aes()`, you need to use `sample`.

The x-axis values correspond to the quantiles of a theoretically normal curve with mean 0 and standard deviation 1 (i.e., the standard normal distribution). The y-axis values correspond to the quantiles of the original unstandardized sample data. However, even if we were to standardize the sample data values, the Q-Q plot would look identical. A data set that is nearly normal will result in a probability plot where the

points closely follow a diagonal line. Any deviations from normality leads to deviations of these points from that line.

The plot for Dairy Queen's calories from fat shows points that tend to follow the line but with some errant points towards the upper tail. You're left with the same problem that we encountered with the histogram above: how close is close enough?

A useful way to address this question is to rephrase it as: what do probability plots look like for data that I *know* came from a normal distribution? We can answer this by simulating data from a normal distribution using `rnorm`.

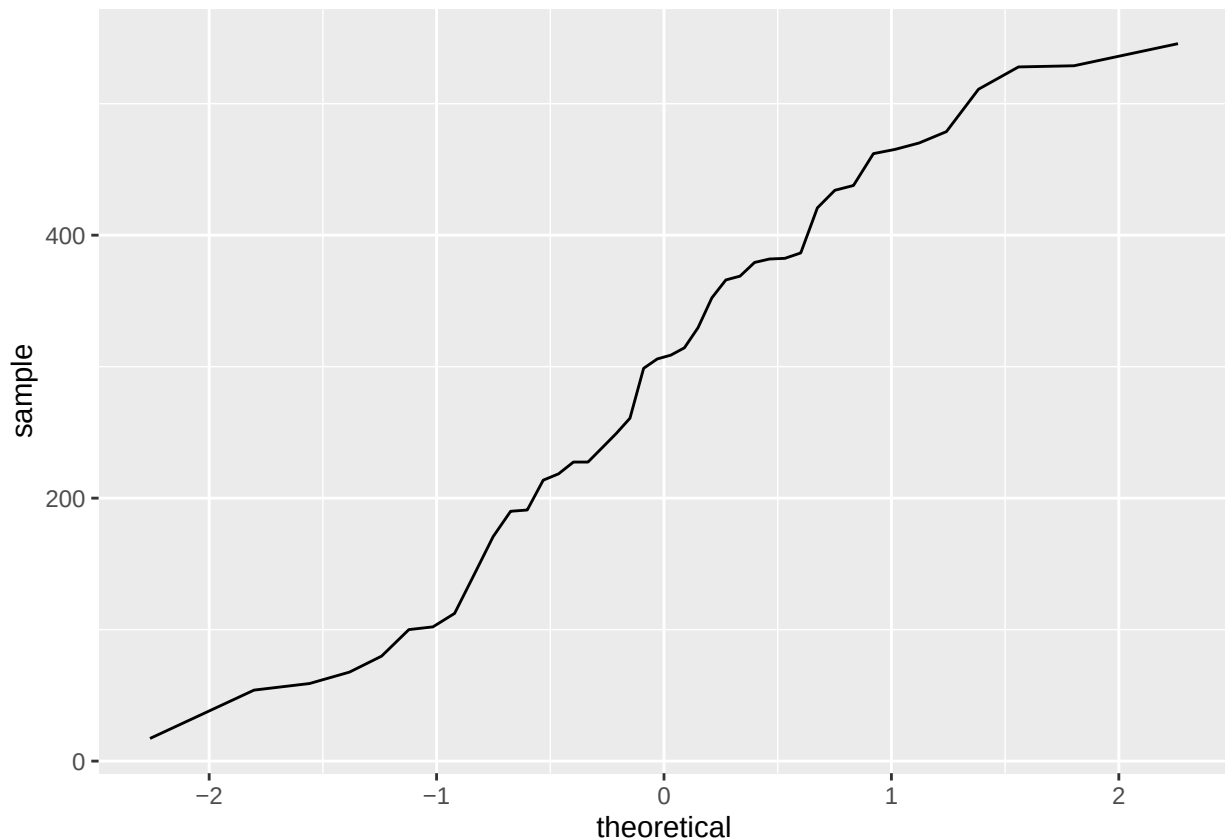
```
sim_norm <- rnorm(n = nrow(dairy_queen), mean = dqmean, sd = dqsd)
```

The first argument indicates how many numbers you'd like to generate, which we specify to be the same number of menu items in the `dairy_queen` data set using the `nrow()` function. The last two arguments determine the mean and standard deviation of the normal distribution from which the simulated sample will be generated. You can take a look at the shape of our simulated data set, `sim_norm`, as well as its normal probability plot.

3. Make a normal probability plot of `sim_norm`. Do all of the points fall on the line? How does this plot compare to the probability plot for the real data? (Since `sim_norm` is not a data frame, it can be put directly into the `sample` argument and the `data` argument can be dropped.)

Insert your answer here

```
ggplot(data = NULL, aes(sample = sim_norm)) +  
  geom_line(stat = "qq")
```

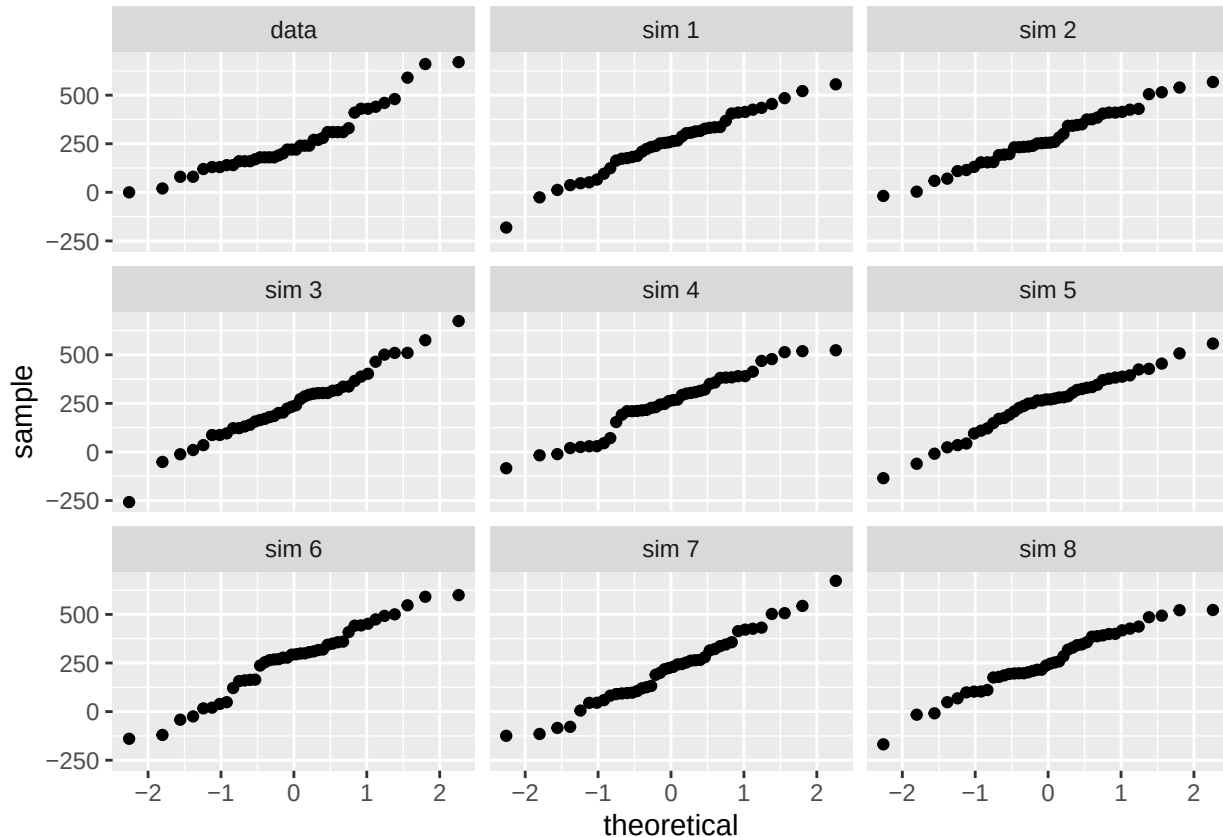


Not all data points fall on the regression line. However, they fit well along it, with slight fluctuations towards the tails. The Dairy Queen data plot has a clear deviation from the regression line as it bends upwards at

the higher end due to right skewness.

Even better than comparing the original plot to a single plot generated from a normal distribution is to compare it to many more plots using the following function. It shows the Q-Q plot corresponding to the original data in the top left corner, and the Q-Q plots of 8 different simulated normal data. It may be helpful to click the zoom button in the plot window.

```
qqnormsim(sample = cal_fat, data = dairy_queen)
```



4. Does the normal probability plot for the calories from fat look similar to the plots created for the simulated data? That is, do the plots provide evidence that the calories are nearly normal?

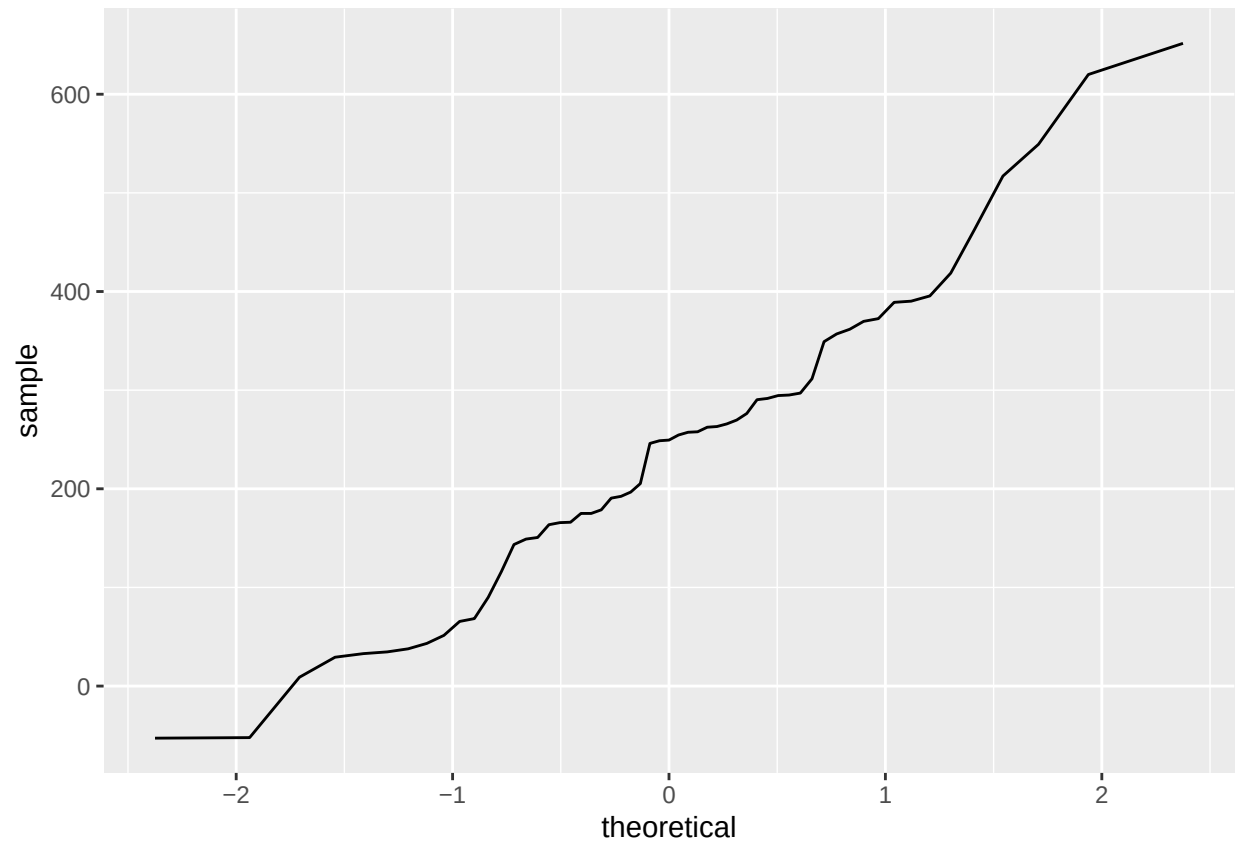
Insert your answer here

What makes the Q-Q plot for the actual data set different is that it deviates from those created by simulation. The simulated data sets appear linear, but the Q-Q plot for Dairy Queen bends upwards in the high-end tail and bends downwards in the low-end tail, where the minimum is 0. It suggests that the calories from fat are not close to being normally distributed.

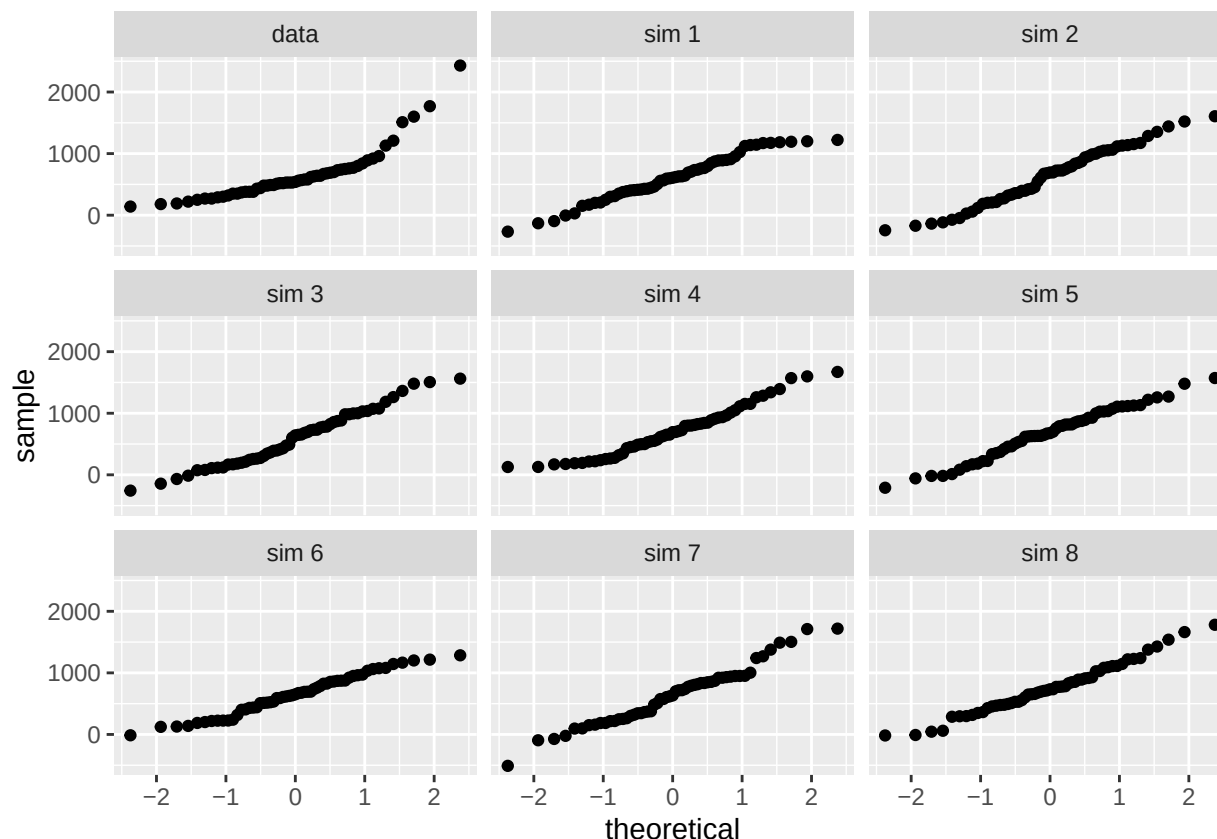
5. Using the same technique, determine whether or not the calories from McDonald's menu appear to come from a normal distribution.

Insert your answer here

```
sim_norm1 <- rnorm(n = nrow(mcdonalds), mean = dqmean, sd = dqsd)
ggplot(data = NULL, aes(sample = sim_norm1)) +
  geom_line(stat = "qq")
```



```
qqnormsim(sample = calories, data = mcdonalds)
```



The calories in McDonald's do not appear normally distributed. While the Q-Q plot for the actual data set curves upwards sharply towards the high values, those in the simulations follow the straight line much more closely. This distribution is very heavy right skewed.

Normal probabilities

Okay, so now you have a slew of tools to judge whether or not a variable is normally distributed. Why should you care?

It turns out that statisticians know a lot about the normal distribution. Once you decide that a random variable is approximately normal, you can answer all sorts of questions about that variable related to probability. Take, for example, the question of, "What is the probability that a randomly chosen Dairy Queen product has more than 600 calories from fat?"

If we assume that the calories from fat from Dairy Queen's menu are normally distributed (a very close approximation is also okay), we can find this probability by calculating a Z score and consulting a Z table (also called a normal probability table). In R, this is done in one step with the function `pnorm()`.

```
1 - pnorm(q = 600, mean = dqmean, sd = dqsd)
```

```
## [1] 0.01501523
```

Note that the function `pnorm()` gives the area under the normal curve below a given value, `q`, with a given mean and standard deviation. Since we're interested in the probability that a Dairy Queen item has more than 600 calories from fat, we have to take one minus that probability.

Assuming a normal distribution has allowed us to calculate a theoretical probability. If we want to calculate the probability empirically, we simply need to determine how many observations fall above 600 then divide this number by the total sample size.

```
dairy_queen %>%
  filter(cal_fat > 600) %>%
  summarise(percent = n() / nrow(dairy_queen))
```

```
## # A tibble: 1 x 1
##   percent
##   <dbl>
## 1 0.0476
```

Although the probabilities are not exactly the same, they are reasonably close. The closer that your distribution is to being normal, the more accurate the theoretical probabilities will be.

6. Write out two probability questions that you would like to answer about any of the restaurants in this dataset. Calculate those probabilities using both the theoretical normal distribution as well as the empirical distribution (four probabilities in all). Which one had a closer agreement between the two methods?

Insert your answer here

Questions:

1. What is the probability that a randomly chosen Dairy Queen item has fewer than 200 calories from fat?
2. What is the probability that a randomly chosen McDonald's item has more than 500 calories?

```
# Q1: Dairy Queen, cal_fat < 200
pnorm(200, mean = dqmean, sd = dqsd)
```

```
## [1] 0.3495757
```

```
dairy_queen %>% summarise(empirical = mean(cal_fat < 200))
```

```
## # A tibble: 1 x 1
##   empirical
##   <dbl>
## 1 0.429
```

```
# Q2: McDonald's, calories > 500
mcmean <- mean(mcdonalds$calories)
mcstd <- sd(mcdonalds$calories)
1 - pnorm(500, mean = mcmean, sd = mcstd)
```

```
## [1] 0.6337263
```

```
mcdonalds %>% summarise(empirical = mean(calories > 500))
```

```
## # A tibble: 1 x 1
##   empirical
##   <dbl>
## 1 0.614
```

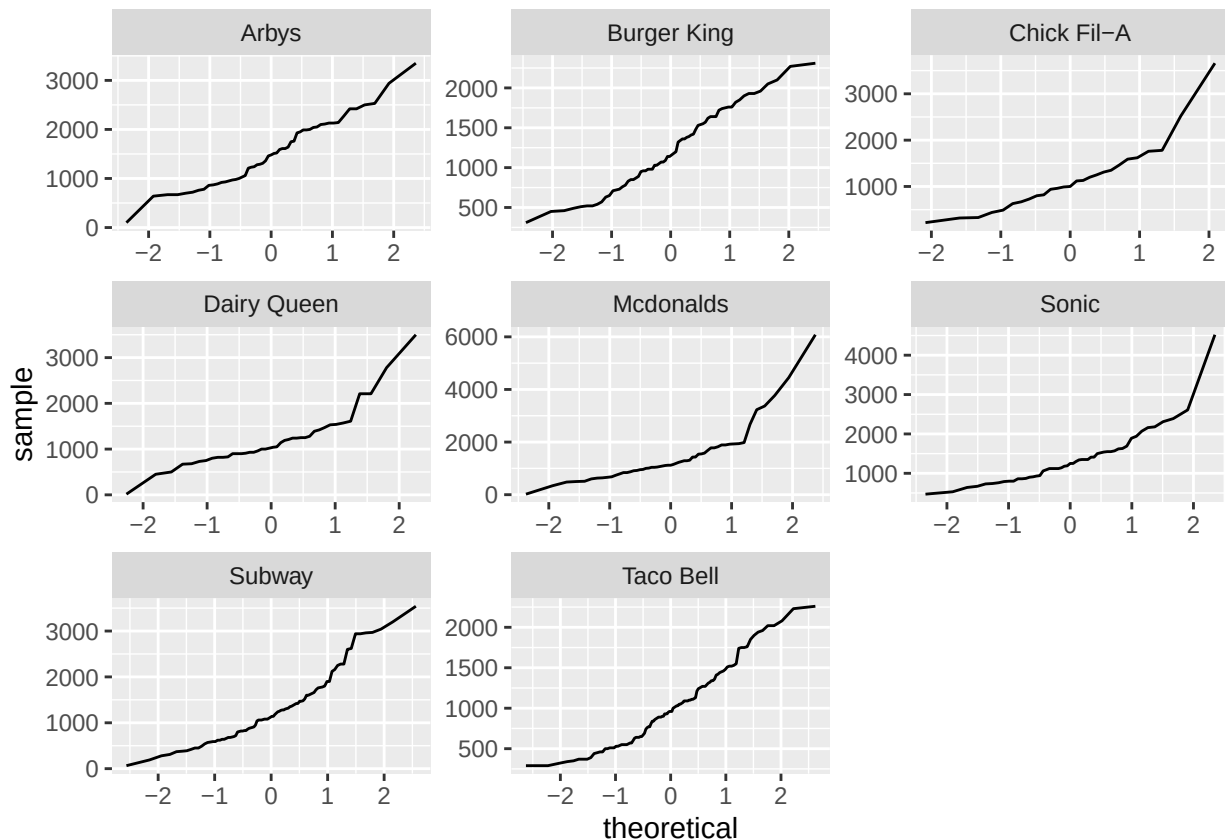
The McDonald's question agrees more closely as it has a difference of about 0.02, vs. about 0.08 for Dairy Queen. This is a coincidence since McDonald's calories are not very normal overall. The threshold of 500 is near the center of the distribution, where the normal curve fits reasonably, while the tails are where a skewed distribution and a normal curve disagree most.

More Practice

7. Now let's consider some of the other variables in the dataset. Out of all the different restaurants, which ones' distribution is the closest to normal for sodium?

Insert your answer here

```
ggplot(fastfood, aes(sample = sodium)) +  
  geom_line(stat = "qq") +  
  facet_wrap(~ restaurant, scales = "free")
```



Burger King is the closest to normal. Its Q-Q plot hugs the line most tightly and its skewness is near zero. Arby's and Taco Bell are the next closest; McDonald's, Sonic, and Dairy Queen are the furthest from normal.

8. Note that some of the normal probability plots for sodium distributions seem to have a stepwise pattern. why do you think this might be the case?

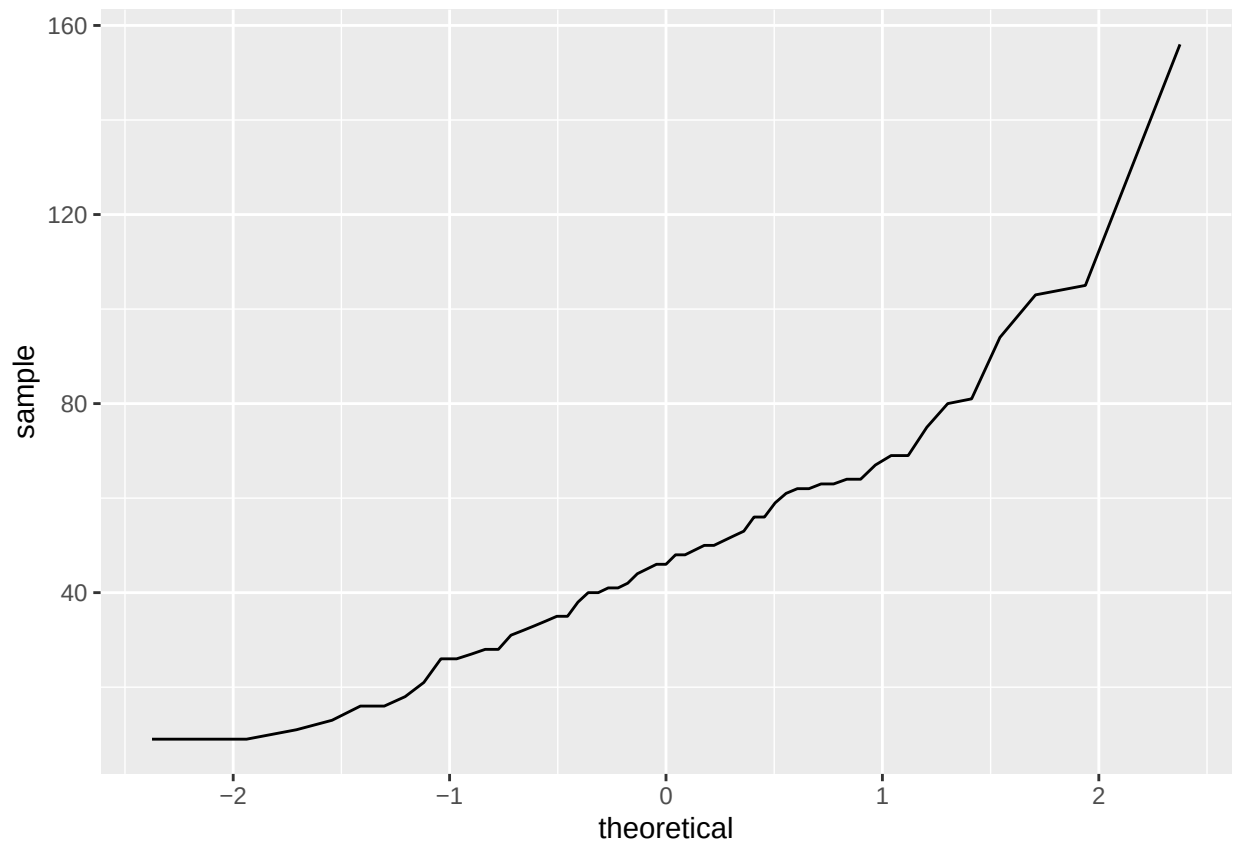
Insert your answer here

The steps occur due to the fact that the sodium content is measured in whole numbers, and many food items have the exact same sodium content. This results in ties, leading to horizontal steps in the plot, as the same quantile in the sample is plotted against several quantiles from the theoretical distribution.

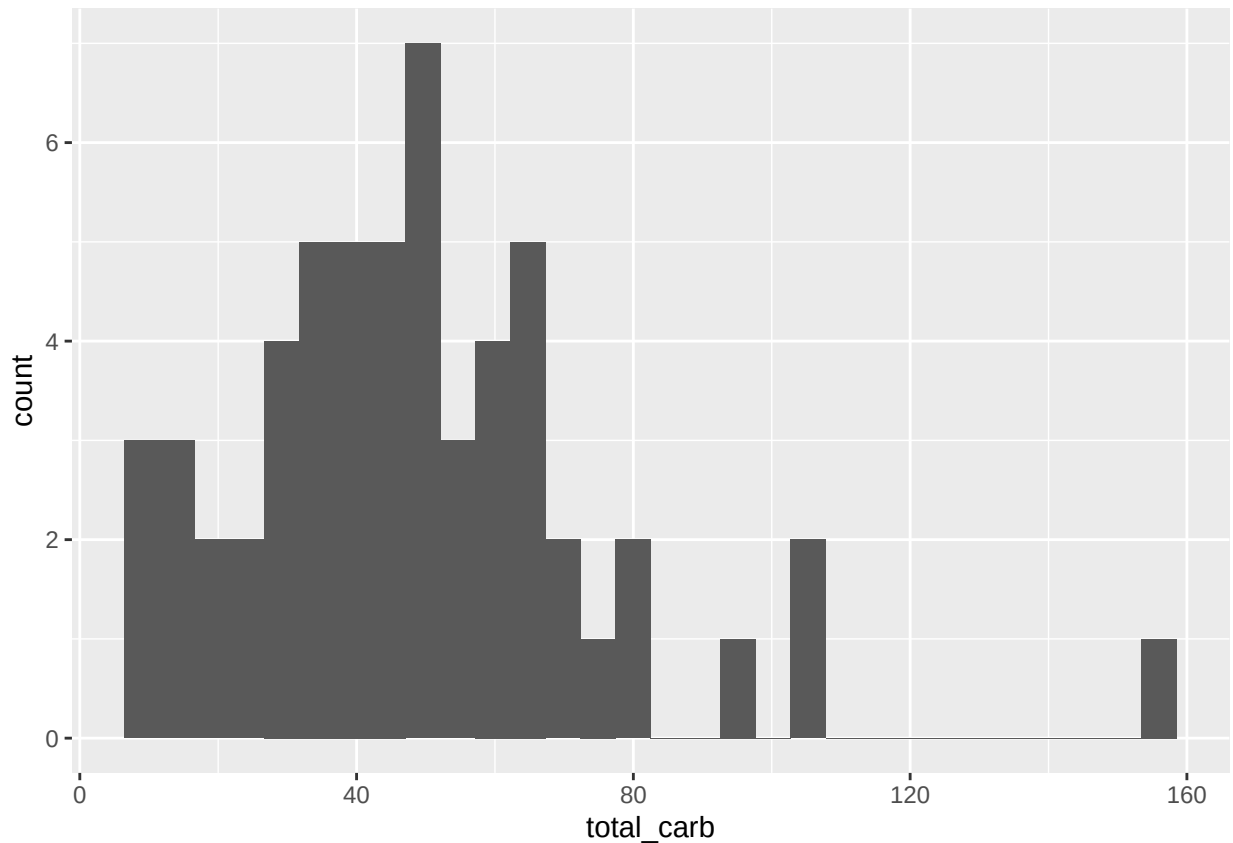
9. As you can see, normal probability plots can be used both to assess normality and visualize skewness. Make a normal probability plot for the total carbohydrates from a restaurant of your choice. Based on this normal probability plot, is this variable left skewed, symmetric, or right skewed? Use a histogram to confirm your findings.

Insert your answer here

```
mcdonalds %>%
  ggplot(aes(sample = total_carb)) +
  geom_line(stat = "qq")
```



```
mcdonalds %>%
  ggplot(aes(x = total_carb)) +
  geom_histogram()
```



```
mcdonalds %>%
  summarise(
    mean = mean(total_carb),
    median = median(total_carb)
  )
```

```
## # A tibble: 1 x 2
##   mean median
##   <dbl> <dbl>
## 1  48.8    46
```

I chose McDonald's. The Q-Q plot curves upward above the line, with the upper points rising far above where a normal would put them, so the variable is **right skewed**. The histogram confirms it: most items cluster at lower carb values with a long tail to the right proven by mean 48.8 > median 46.